

Note that Scala can't create arrays for every type *A*. It requires that a `Manifest[A]` exists in implicit scope. Scala's standard library provides manifests for most types that you would in practice want to put in an array.

Just like with `STRef`, we always return an `STArray` packaged in an `ST` action with a corresponding *S* type, and any manipulation of the array (even reading it) is an `ST` action tagged with the same type *S*. It's therefore impossible to observe a naked `STArray` outside of the `ST` monad (except by code in the Scala source file in which the `STArray` data type itself is declared).

Using these primitives, we can write more complex functions on arrays.



EXERCISE 14.1

Add a combinator on `STArray` to fill the array from a `Map` where each key in the map represents an index into the array, and the value under that key is written to the array at that index. For example, `xs.fill(Map(0->"a", 2->"b"))` should write the value "a" at index 0 in the array `xs` and "b" at index 2. Use the existing combinators to write your implementation.

```
def fill(xs: Map[Int,A]): ST[S,Unit]
```

Not everything can be done efficiently using these existing combinators. For example, the Scala library already has an efficient way of turning a list into an array. Let's make that primitive as well:

```
def fromList[S,A:Manifest](xs: List[A]): ST[S, STArray[S,A]] =
  ST(new STArray[S,A] {
    lazy val value = xs.toArray
  })
```

14.2.5 A purely functional in-place quicksort

The components for quicksort are now easy to write in `ST`—for example, the swap function that swaps two elements of the array:

```
def swap[S](i: Int, j: Int): ST[S,Unit] = for {
  x <- read(i)
  y <- read(j)
  _ <- write(i, y)
  _ <- write(j, x)
} yield ()
```



EXERCISE 14.2

Write the purely functional versions of `partition` and `qs`.